



回溯法

《算法分析与设计》

授课人：罗建超、长聘副教授、博导

日期：2025年11月14日

目录

1

- 回溯法及其基本思想

2

- 回溯法解题算法框架

3

- 案例分析：N皇后问题

4

- 案例分析：货郎担问题

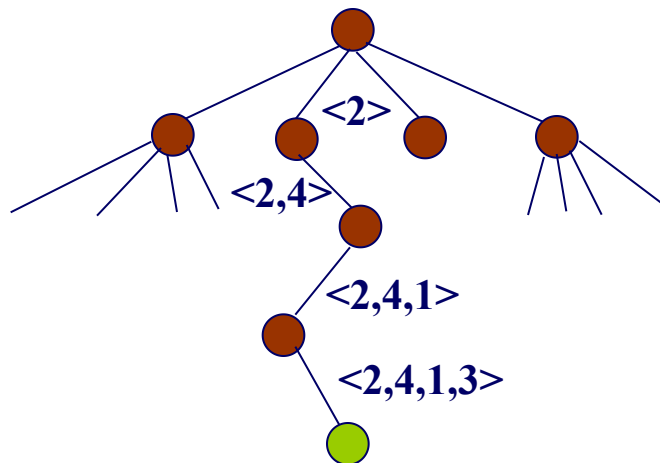
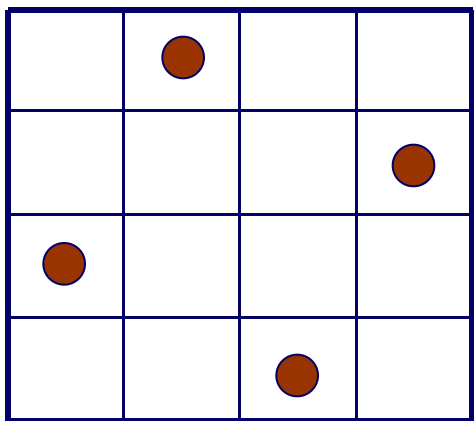
5

- 其它案例分析

问题回顾

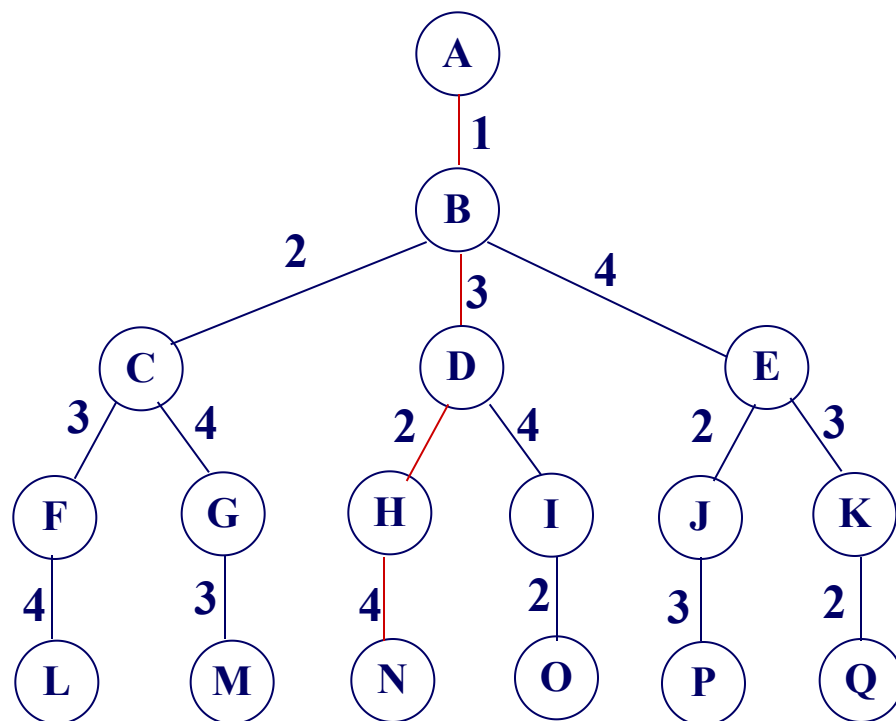
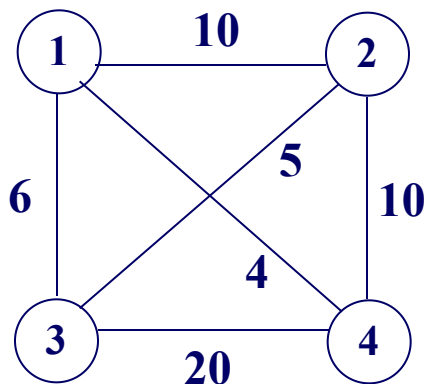
N后问题:

在 $N \times N$ 的棋盘上放置 N 个皇后，使得任何两个皇后之间不能相互攻击（不在同一行，同一列，同一对角线），试给出所有的放置方法。



问题回顾

旅行售货员（货郎担）问题：某售货员要到若干城市去推销商品，已知各城市间的路程耗费（代价），如何选定一条从驻地出发，经过每个城市一遍，最后回到驻地的路线，使得总路程耗费最小。



回溯法：基本思想

◆ 回溯法也称**试探法**，其**基本思想**是：

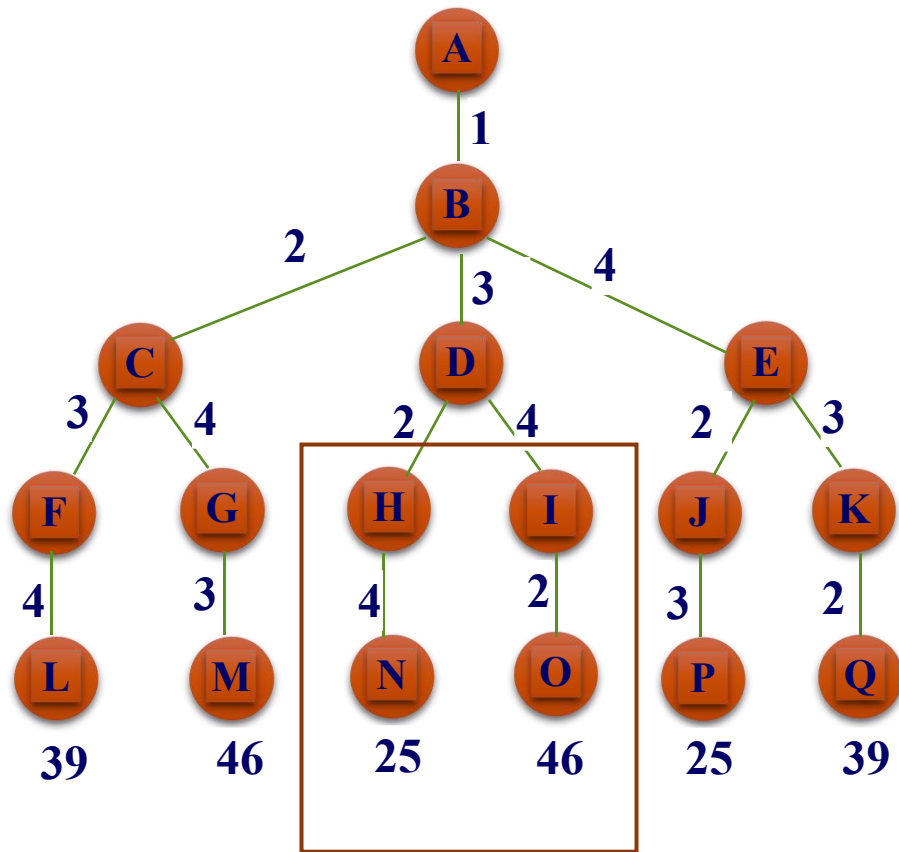
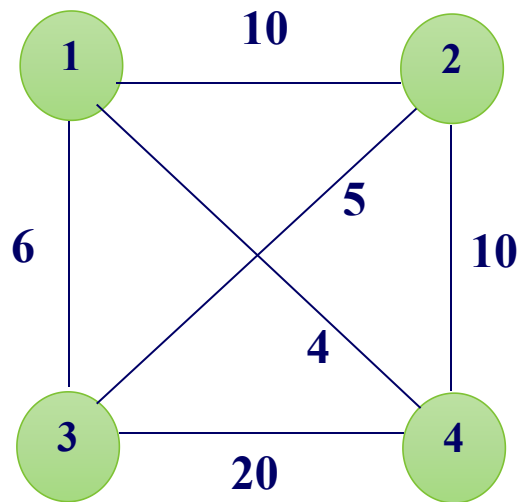
◆ 从初始状态出发，搜索其所能到达的所有“状态”；

◆ 当一条路走到尽头，再**后退一步或若干步**，从另外一种状态出发，继续搜索，直到所有的路径都搜索过。

◆ 这种不断**前进**、不断**回溯**寻找解的方法叫**回溯法**。

回溯法：基本思想

举例分析：TSP问题回溯法求解



回溯法：重点与难点

◆ 回溯法通常将问题解空间组织成“**树**”结构，采用系统的方法**搜索**解空间树，从而得到问题解。

◆ **搜索策略**：深度优先**为主**，也可以采用广度优先、函数优先、广度深度结合等。

◆ **避免无效搜索策略**：

- **约束函数**：在扩展结点处剪去不满足约束条件的子树
- **界限函数**：在扩展结点处剪去得不到最优解的子树

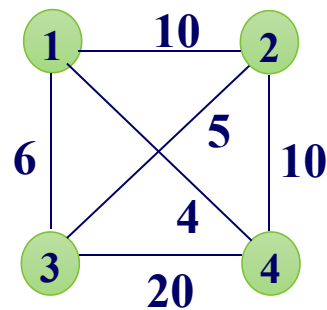
回溯法:重点与难点

◆ **问题解的表示**: 将问题的解表示成一个 n 元式 $(x_1, x_2, \dots, x_n)$ 的形式。

◆ **显示约束**: 对分量 x_i 的取值限定。

◆ **隐示约束**: 为满足问题的解而对不同分量之间施加的约束。

◆ **解空间**: 解向量满足约束条件的所有多元组, 构成了问题的一个解空间



◆ **问题解的表示**: $(x_1, x_2, \dots, x_4)$

◆ **显示约束**: $x_i \in \{1, 2, 3, 4\}$ 。

◆ **隐示约束**: $(x_i, x_{i+1}) \in E$ 和 $\forall i, k \in \{1, 2, 3, 4\}, x_i \neq x_k, (x_n, x_1) \in E$

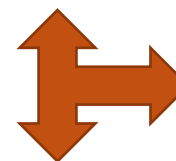
◆ **解空间**: 满足 $x_i \in \{1, 2, 3, 4\}$, $(x_i, x_{i+1}) \in E, \forall i, k \in \{1, 2, 3, 4\}, x_i \neq x_k$, 且 $(x_n, x_1) \in E$ 的所有 $(x_1, x_2, \dots, x_4)$ 。



回溯法相关概念

◆ 结点状态：

● 白结点（尚未访问）



活结点

● 灰结点（正在访问以该结点为根的子树）

● 黑结点（以该结点为根的子树遍历完成）



死结点

◆ 存储：当前路径

回溯法适用条件

回溯法适用问题： 搜索问题和优化问题。

必要条件： 多米诺性质

设 $P(x_1, x_2, \dots, x_i)$ 是关于向量 $\langle x_1, x_2, \dots, x_i \rangle$ 的某个性质，那么 $P(x_1, x_2, \dots, x_{i+1})$ 真蕴含 $P(x_1, x_2, \dots, x_i)$ 为真，即

$$P(x_1, x_2, \dots, x_{i+1}) \rightarrow P(x_1, x_2, \dots, x_i) \quad (0 < i < n) \quad (n \text{ 为向量维数})$$

回溯法适用条件

例如：求满足不等式的所有整数解问题

$$5x_1 + 4x_2 - x_3 \leq 10 \quad x_i \in \{1, 2, 3\}, i = 1, 2, 3$$

不满足多米诺性质

$\langle x_1, x_2, x_3 \rangle = \langle 1, 2, 3 \rangle$, $P(x_1, x_2, x_3)$ 为真, 但 $P(x_1, x_2)$ 为假。

导致的问题：结点 $\langle 1, 2 \rangle$ 被误剪枝, 导致结点 $\langle 1, 2, 3 \rangle$ 不可达, 从而不能确保所有的解。

解决办法2：变量替换。替换为 $5x_1 + 4x_2 + (x_4 - 4) \leq 10$, $x_4 \in \{1, 2, 3\}$, $5x_1 + 4x_2 + x_4 \leq 14$; $x_3 = 4 - x_4$;

回溯法设计要素

◆ 针对问题定义解空间：

➤ 问题解向量

➤ 解向量分量取值集合

➤ 构造解空间树

◆ 判断问题是否满足多米诺性质

◆ 搜索解空间树，确定剪枝函数

◆ 确定存储搜索路径的数据结构

两类典型的解空间树

◆ **子集树**：当所给的问题是从 n 个元素的集合 S 中找出满足某种性质的**子集**时，相应的解空间树称为子集树。子集树通常有 2^n 个叶结点。（**01背包问题**）

◆ **排列树**：当所给的问题是确定 n 个元素满足某种性质的**排列**时，相应的解空间树称为排列树。排列树通常有 $n!$ 个叶结点。（**货郎担问题**）

目录

1

- 回溯法及其基本思想

2

- 回溯法解题算法框架

3

- 案例分析：N皇后问题

4

- 案例分析：货郎担问题

5

- 其它案例分析

两种解题框架

◆ 递归回溯框架

```
void backtrack (int t)
{
    if (t > n) output(x);
    else
        for (int i = f(n, t); i ≤ g(n, t); i++)
        {
            x[t] = h(i);
            if (constraint(t) && bound(t))
                backtrack(t+1);
        }
}
```

t : 递归深度;

n : 最大递归深度;

x : 解向量

output(x): 输出得到的可行解 x ;

$f(n, t)$, $g(n, t)$: 当前扩展结点处未搜索的子树的起始编号和终止编号;

$h(i)$: 当前扩展结点 $x[t]$ 的第 i 个可选值;

constraint(t), bound(t): 当前扩展结点的约束函数, 界函数;

两种解题框架

◆ 迭代回溯框架

```
void iterativeBacktrack ()
{
    int t = 1;
    while (t > 0) {//退到无路可退，结束
        if (f(n, t) ≤ g(n, t))
            for (int i = f(n, t); i ≤ g(n, t); i++) {
                x[t] = h(i);
                if (constraint(t) && bound(t)) {
                    if (solution(t)) output(x);
                    else t++; //等价于backtrack(t+1)
                }
            }
        else t--; //走不通了后退
    }
}
```

t : 迭代深度;

x : 解向量;

output(x): 输出得到的可行解 x ;

$f(n, t)$, $g(n, t)$: 当前扩展结点处未搜索的子树的**起始编号**和**终止编号**;

$h(i)$: 当前扩展结点 $x[t]$ 的第 i 个可选值;

$\text{constraint}(t)$, $\text{bound}(t)$: 当前扩展结点的**约束函数**, **界函数**;

solution(t): 判断当前扩展结点处是否得到问题的可行解。

回溯法的优点

在搜索过程中动态产生问题的解空间。在任何时刻，算法只保存从根结点到扩展结点的路径，如果解空间树中从根结点到叶子结点的最长路径的长度为 $h(n)$ ，则回溯法的空间复杂度通常为 $O(h(n))$ 。显示存储整个解空间则需要 $O(m^{h(n)})$ 或者 $O(h(n)!)$ 。

目录

1

- 回溯法及其基本思想

2

- 回溯法解题算法框架

3

- 案例分析：N皇后问题

4

- 案例分析：货郎担问题

5

- 其它案例分析

N皇后问题

N皇后问题

在 $n \times n$ 的棋盘上放置 n 个皇后，使得任何两个皇后之间不能相互攻击，试给出**所有**的放置方法。

1				Q				
2						Q		
3								Q
4		Q						
5							Q	
6	Q							
7			Q					
8					Q			
	1	2	3	4	5	6	7	8

N皇后问题

- 1) 每一行有且只有一个皇后，**设第 i 行的皇后所在的列为 x_i** ，则问题解向量： $(x_1, x_2, \dots, x_n)$
- 2) 显约束： $x_i = 1, 2, \dots, n$
- 3) 隐约束：
 - (1) 不同列： $x_i \neq x_j$;
 - (2) 不处于同一正 ($x_i - i \neq x_j - j$)、反对角线 ($x_i + i \neq x_j + j$) : **合并后得到 $|i - j| \neq |x_i - x_j|$ 。**

N皇后问题

```
Queen1() {  
    int x[9];  
    for x[1] = 1:8  
        for x[2] = 1:8  
            if(check(x, 2) = 0) continue;  
            for x[3] = 1: 8  
                if(check(x, 3) = 0) continue;  
                for x[8] = 1: 8  
                    if(check(x, 8) = 0) continue;  
                    else print(x);}
```

缺点：不适用于规模N的问题

只需check新放的皇后和之前的皇后是否冲突，之前的皇后之间不需要判断

```
check(int x[], int pos) {  
    for(int i = 1; i < pos; i++)  
        if(abs(x[i] - x[pos]) == abs (i - pos) or x[i] == x[pos])  
            return 0;  
    return 1; }
```

Queen1算法分析

Queen1算法的时间复杂度上界为 $O(8^9)$ ，其实算法在执行过程中不断地运行到continue语句，所以算法的复杂度远低于 8^9 。

把continue和check都去掉，只在最后一步判断是否满足约束条件，则问题的复杂度就变为了 8^9 。

N皇后问题—递归求解

```
int n = *; sum = 0; x[n];  
void Queen::Backtrack(int t) {  
    if (t > n) sum++;  
    else  
        for (int i = 1; i ≤ n; i++) {  
            x[t] = i;  
            if (check(x, t)) Backtrack(t+1);  
        }  
}
```

```
check(int x[], int pos) {  
    for(int i = 1; i < pos; i++)  
        if(abs(x[i] - x[pos]) == abs (i - pos) or x[i] == x[pos])  
            return 0;  
    return 1; }
```

递归模型与递归解题框架的对比

```
void backtrack (int t)
{
    if ( $t > n$ ) output(x);
    else
        for (int  $i = f(n, t) : g(n, t)$ )
        {
             $x[t] = h(i)$ ;
            if (constraint( $t$ ) && bound( $t$ )) backtrack( $t+1$ );
        }
}
```

```
void Queen::Backtrack(int t) {
    if ( $t > n$ ) sum++;
    else
        for (int  $i = 1$ ;  $i \leq n$ ;  $i++$ ) {
             $x[t] = i$ ;
            if (check( $x, t$ )) Backtrack( $t+1$ );
        }
}
```


N皇后问题—迭代求解

```
void iterativeBacktrack ()
{
    int t = 1;
    while (t > 0) {
        if ( $f(n, t) \leq g(n, t)$ )
            for ( $i = f(n, t) : g(n, t)$ ) {
                 $x[t] = h(i)$ ;
                if (constraint(t)&&bound(t)) {
                    if (solution(t)) output(x);
                    else t++;
                }
            }
        else t--;
    }
}
```

```
void iterativeBacktrack (int n)
{
    int t = 1;
    int[] f; // 记录每一层皇后尝试到哪了
    while (t > 0) {
        if (f[t] ≤ n)
            for (; f[t] ≤ n; f[t]++) {
                 $x[t] = f[t]$ ;
                if (check(x, t))
                    if (t = n) output(x);
                else
                    t++;
            }
        else t--;
    }
}
```

N皇后问题—迭代求解

```
void iterativeBacktrack (int n)
{
    int t = 1;
    int[] f; // record its initial value
    while (t > 0) {
        if (f[t] ≤ n)
            for (; f[t] ≤ n; f[t]++) {
                x[t] = f[t];
                if (check(x, t))
                    if (t == n) output(x);
                else
                    t++;
            }
        else t--;
    }
}
```

t = 1;

f[t] = {0, 1, 1, 1, 1};

t = 2;

f[t] = {0, 1, 3, 1, 1};

●			
		●	

t = 3;

f[t] = {0, 1, 3, 5, 1};

●			
		●	

t = 2;

f[t] = {0, 1, 3, 5, 1};

N皇后问题—迭代求解

```
void iterativeBacktrack (int n)
{
    int t = 1;
    int[] f; // record its initial value
    while (t > 0) {
        if (f[t] ≤ n)
            for (; f[t] ≤ n; f[t]++) {
                x[t] = f[t];
                if (check(x, t))
                    if (t == n) output(x);
                else
                    t++;
            }
        else {
            t--;
            f[t]++;
        }
    }
}
```

●			
		●	

t = 3;
f[t] = {0, 1, 3, 5, 1};

●			
			●

t = 2;
f[t] = {0, 1, 4, 5, 1};

●			
		●	

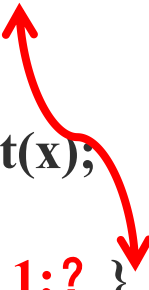
t = 2;
f[t] = {0, 1, 4, 5, 1};

●			
			●

t = 3;
f[t] = {0, 1, 4, 5, 1};

N皇后问题—迭代求解

```
void iterativeBacktrack (int n)
{
    int t = 1;
    int[] f;// record its initial value
    while (t > 0) {
        if(f[t] ≤ n)
            for(; f[t] ≤ n; f[t]++) {
                x[t] = f[t];
                if(check(x, t))
                    if (t==n)output(x);
                else {
                    t++; f[t] = 1; ? }
            }
        else {
            f[t] = 1; ?
            t--;
            f[t]++;
        }
    }
}
```



N皇后问题—迭代求解

```
void iterativeBacktrack (int n)
{
    int t = 1;
    while (t > 0) {
        a[t]++;
        while(a[t] <= n && !check(t, a))
            a[t]++;
        if (a[t] <= n) {//找到一个可以放置的位置
            if (t == n) output(a);
            else {t++; a[t] = 0} }
        else t--;
    }
}
```

```
}
check(int pos, int a[]) {
    for(int i = 1; i < pos; i++)
        if(abs(a[i] - a[pos]) == abs (i - pos) or a[i] == a[pos])
            return 0;
    return 1; }
```

迭代模型与迭代解题框架的对比

```
void iterativeBacktrack ()
```

```
{
    int t = 1;
    while (t > 0) {
        if ( $f(n, t) \leq g(n, t)$ )
            for (int i = f(n, t); i ≤ g(n, t); i++) {
                x[t] = h(i);
                if (constraint(t) && bound(t)) {
                    if (solution(t)) output(x);
                    else t++;
                }
            }
        else t--;
    }
}
```

```
void iterativeBacktrack (int n)
```

```
{
    int t = 1;
    int[] f; // record its initial value
    while (t > 0) {
        if (f[t] ≤ n)
            for (; f[t] ≤ n; f[t]++) {
                x[t] = f[t];
                if (check(x, t))
                    if (t == n) output(x);
                else {
                    t++; f[t] = 0;
                }
            }
        else {
            t--;
            f[t]++;
        }
    }
}
```

目录

1

- 回溯法及其基本思想

2

- 回溯法解题算法框架

3

- 案例分析：N皇后问题

4

- 案例分析：货郎担问题

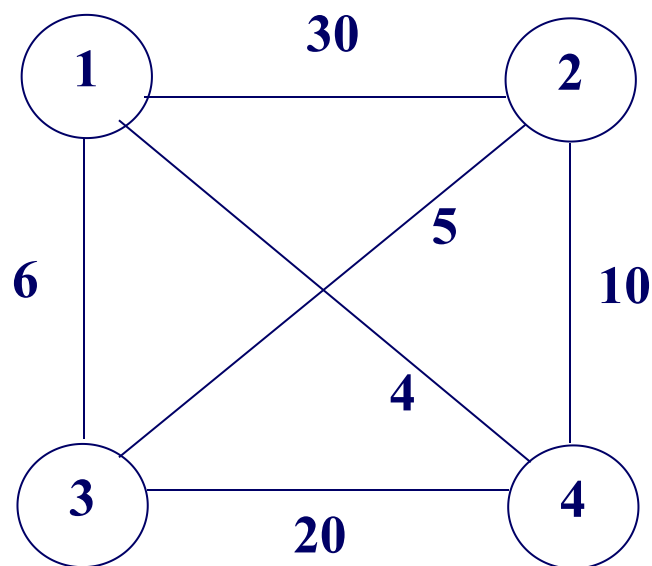
5

- 其它案例分析

货郎担问题

旅行售货员（货郎担）问题：

某售货员要到若干城市去推销商品，已知各城市间的路程耗费（代价），如何选定一条从驻地出发，经过**每个城市一遍**，最后回到驻地的路线，使得**总路程耗费**最小。



货郎担问题：解空间表示

- 1) 每个城市只出现有且仅有一次，设第 i 个出现的城市为 x_i ，则问题解向量： $(x_1, x_2, \dots, x_n)$
- 2) 显约束： $x_i = 1, 2, \dots, n$
- 3) 隐约束：
 - (1) 有从 x_i 到 x_{i+1} 的边；
 - (2) 有从 x_n 到 x_1 的边； //能回到出发城市
 - (3) $x_i \neq x_k$ ； //城市不能重复

货郎担问题—递归求解

```
void backtrack (int t)
{
    if (t > n) output(x);
    else
        for (int i = f(n, t) : g(n, t))
        {
            x[t] = h(i);
            if (constraint(t) & bound(t))
                backtrack(t+1);
        }
}
```

```
a[n][n]; //邻接矩阵, 存储任意两个城市间的代价;
bestx[n]; //存储当前最小代价对应的路线;
bestc = MaxInt; //存储当前最小代价
void Traveling<Type>::Backtrack(int t){ //t的初值为2;
    if (t > n)
        if (cost(x) < bestc) {
            bestc = cost(x); bestx = x;
        }
    else
        for (int j = 1; j ≤ n; j++)
        {
            x[t] = j;
            if (check(x, t, a, n))
                backtrack(t + 1);
        }
}
```

深度为n时，原本只有1个城市可选了，却要做1到n的循环，且每次循环还需要花O(n)做重复性检测！

```
check (int[] x, int t, int[][] a, int n) {
    if (t < 2) return 1;
    if (noRepeated(x, t)) { //重复性检查
        if (t < n && a[x[t-1]][x[t]] != NoEdge) return 1;
        if (t = n && a[x[t-1]][x[t]] != NoEdge && a[x[t]][x[1]] != NoEdge ) return 1; }
    else return 0;}
```

货郎担问题—递归求解

$x[] = \{*, 1, 2, 3, 4, 5\}$

$t = 2$

$x[] = \{*, 1, 2, 3, 4, 5\}$ 将 $x[2]$ - $x[5]$ 依次交换到 $x[2]$

假设 $t = 2$ 时，将 $x[3]$ 交换到 $x[2]$ ，且check通过

$t = 3$

$x[] = \{*, 1, 3, 2, 4, 5\}$ 将 $x[3]$ - $x[5]$ 依次交换到 $x[3]$

依次类推，深度 t 的时候，只需要考虑 $x[t]$ - $x[n]$ 的城市

货郎担问题—递归求解

```
a[n][n]; //邻接矩阵, 存储任意两个城市间的代价;
bestx[n]; //存储当前最小代价对应的路线;
bestc = MaxInt; //存储当前最小代价
x[] = {0, 1, 2, 3, ..., n}
void Traveling<Type>::Backtrack(int t){ //t的初值为2;
    if (t > n) {
        if(cost(x) < bestc) {
            bestc = cost(x); bestx = x;
        }
    }
    else {
        for (int j = t; j ≤ n; j++){
            if(check(x, j, t, a, n)) { //x需要赋初值包含所有城市
                swap(x[t], x[j]);
                backtrack(t + 1);
                swap(x[t], x[j]); //恢复现场
            }
        }
    }
}
```

算法时间复杂度:
算法backtrack在最坏情况下可能需要更新当前最优解 $O((n-1)!)$ 次, 每次更新bestx需计算时间 $O(n)$, 从而整个算法的计算时间复杂性为 $O(n!)$ 。

别忘了恢复现场

```
check (int[] x, int j, int t, int[][] a, int n) {
    if(t < 2) return 1;
    if(t < n && a[x[t-1]][x[j]] != NoEdge) return 1;
    if(t = n && a[x[t-1]][x[j]] != NoEdge && a[x[j]][x[1]] != NoEdge ) return 1; }
return 0;}
```

Repeated检查没有了



货郎担问题—递归求解

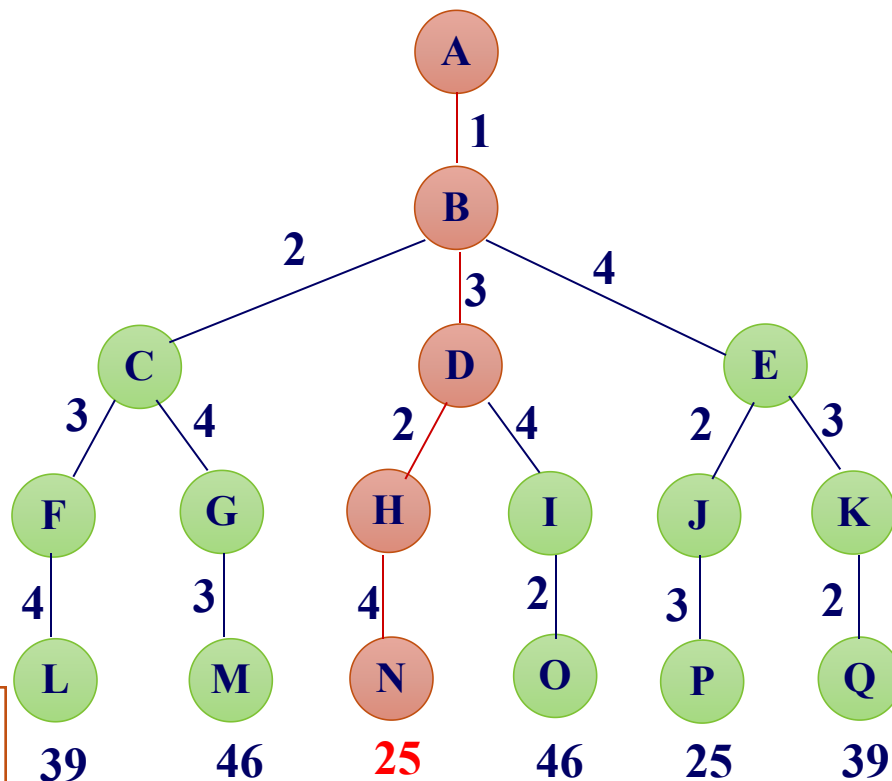
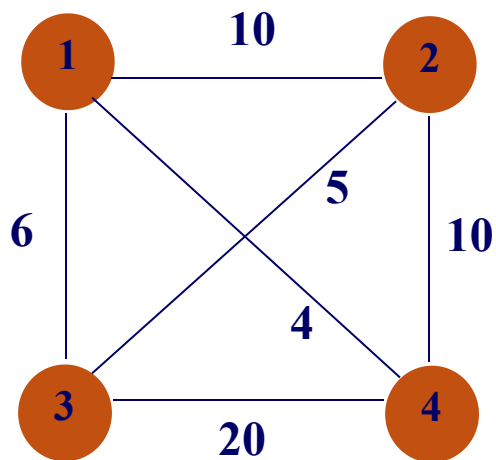
```
a[n][n]; //邻接矩阵, 存储任意两个城市间的代价;
bestx[n]; //存储当前最小代价对应的路线;
bestc = MaxInt; //存储当前最小代价
x[] = {0, 1, 2, 3, ..., n}
void Traveling<Type>::Backtrack(int t){ //t的初值为2;
    if (t > n) {
        if (cost(x) < bestc) {
            bestc = cost(x); bestx = x;
        }
    }
    else {
        for (int j = t; j ≤ n; j++){
            swap(x[t], x[j]);
            if (check(x, t, a, n)) {
                backtrack(t + 1);
            }
            swap(x[t], x[j]);
        }
    }
}
```

算法时间复杂度：
算法backtrack在最坏情况下可能需要更新当前最优解 $O((n-1)!)$ 次，每次更新bestx需计算时间 $O(n)$ ，从而整个算法的计算时间复杂性为 $O(n!)$ 。

```
check (int[] x, int t, int[][] a, int n) {
    if(t < 2) return 1;
    if(t < n && a[x[t-1]][x[t]] != NoEdge) return 1;
    if(t = n && a[x[t-1]][x[t]] != NoEdge && a[x[t]][x[1]] != NoEdge ) return 1;
    return 0;}

```

Repeated检查没有了



找到N点，当前代价最小的路径为**25**，回退到D点，搜索得到I点，A到I的代价为**26**。问：是否还有必要搜索I结点？

货郎担问题—递归求解

```
a[n][n]; //邻接矩阵, 存储任意两个城市间的代价;  
bestx[n]; //存储当前最小代价对应的路线;  
bestc = MaxInt; //存储当前最小代价
```

```
cc = 0; //存储当前代价
```

```
void Traveling<Type>::Backtrack(int t){ //t的初
```

```
    if (t > n) {  
        bestc = cc; bestx = x; }  
    else {
```

```
        for (int j = t; j ≤ n; j++) {  
            if (check(x, j, t, a, n)) {
```

```
                swap(x[t], x[j]);  
                if (t < n && cc + a[x[t-1]][x[t]] < bestc) {
```

```
                    cc = cc + a[x[t-1]][x[t]];
```

```
                    backtrack(t + 1);
```

```
                    cc = cc - a[x[t-1]][x[t]];
```

```
                }
```

```
                if (t == n && cc + a[x[t-1]][x[t]] + a[x[n]][x[1]] < bestc) {
```

```
                    cc = cc + a[x[t-1]][x[t]] + a[x[n]][x[1]];
```

```
                    backtrack(t + 1);
```

```
                    cc = cc - a[x[t-1]][x[t]] - a[x[n]][x[1]];
```

```
                }
```

```
                swap(x[t], x[j]); //恢复现场
```

```
    }  
}
```

算法时间复杂度：
算法backtrack在最坏情况下可能需要更新当前最优解 $O((n-1)!)$ 次，每次更新bestx需计算时间 $O(n)$ ，从而整个算法的计算时间复杂性为 $O(n!)$ 。

货郎担问题—递归求解

```
a[n][n]; //邻接矩阵, 存储任意两个城市间的代价;
cc = 0; 存储当前代价;
bestc = NoEdge; //存储当前最小代价
bestx[n]; 存储当前最小代价对应的路线;
void Traveling<Type>::Backtrack(int t){ //t的初值为2;
    if (t == n) {
        if (a[x[n-1]][x[n]] != NoEdge && a[x[n]][1] != NoEdge &&
            (cc + a[x[n-1]][x[n]] + a[x[n]][1] < bestc || bestc == NoEdge))
            for (int j = 1; j ≤ n; j++) bestx[j] = x[j];
            bestc = cc + a[x[n-1]][x[n]] + a[x[n]][1];
    }
    else { //t < n
        for (int j = t; j ≤ n; j++)
            if (a[x[t-1]][x[j]] != NoEdge && // 是否可进入x[j]子树?
                (cc + a[x[t-1]][x[j]] < bestc || bestc == NoEdge)) {
                // 搜索子树
                Swap(x[t], x[j]);
                cc += a[x[t-1]][x[t]];
                Backtrack(t+1);
                cc -= a[x[t-1]][x[t]]; //恢复现场
                Swap(x[t], x[j]);
            }
    }
}
```

算法时间复杂度：
算法backtrack在最坏情况下可能需要更新当前最优解 $O((n-1)!)$ 次，每次更新bestx需计算时间 $O(n)$ ，从而整个算法的计算时间复杂性为 $O(n!)$ 。

递归求解方法与递归求解框架对比

```
a[n][n]; //邻接矩阵, 存储任意两个城市间的代价;
bestx[n]; //存储当前最小代价对应的路线;
bestc = MaxInt; //存储当前最小代价
cc = 0; //存储当前代价
void Traveling<Type>::Backtrack(int t){ //t的初值为2;
    if (t > n) {
        bestc = cc; bestx = x;}
    else {
        for (int j = t; j ≤ n; j++){
            if(check(x, j, t, a, n) ) {
                swap(x[t], x[j]);
                if(t < n && cc + a[x[t-1]][x[t]] < bestc) {
                    cc = cc + a[x[t-1]][x[t]]; //加入城市x[t]后更新cc
                    backtrack(t + 1);
                    cc = cc - a[x[t-1]][x[t]];
                }
                if(t == n && cc + a[x[t-1]][x[t]] + a[x[n]][x[1]] < bestc) {
                    cc = cc + a[x[t-1]][x[t]] + a[x[n]][x[1]];
                    backtrack(t + 1);
                    cc = cc - a[x[t-1]][x[t]] - a[x[n]][x[1]];
                }
                swap(x[t], x[j]); //恢复现场
            }
        }
    }
}
```

```
void backtrack (int t)
{
    if (t > n) output(x);
    else
        for (int i = f(n, t): g(n, t))
        {
            x[t] = h(i);
            if (constraint(t)
                && bound(t))
                backtrack(t+1);
        }
}
```

货郎担问题—迭代求解



目录

1

- 回溯法及其基本思想

2

- 回溯法解题算法框架

3

- 案例分析：N皇后问题

4

- 案例分析：货郎担问题

5

- 其它案例分析

装载问题

有一批共 n 个集装箱要装上2艘载重量分别为 c_1 和 c_2 的轮船，其中集装箱 i 的重量为 w_i ，且 $\sum_{i=1}^n w_i \leq c_1 + c_2$ 。

装载问题要求确定是否有一个合理的装载方案可将这批集装箱装上这2艘轮船。如果有，找出一种装载方案。

例如：当 $n = 3$, $c_1 = c_2 = 50$, 且 $w = [10, 40, 40]$ 时，可将1和2装上第一艘轮船，3装入第二艘轮船；若 $w = [20, 40, 40]$ ，则无法将这三个集装箱都装上轮船。

装载问题

容易证明，如果一个给定装载问题有解，则采用下面的策略可得到一种装载方案。

(1) 首先将**第一艘轮船尽可能装满**；

(2) 将剩余的集装箱装上第二艘轮船。

将第一艘轮船尽可能装满等价于选取全体集装箱的一个子集，使该子集中集装箱重量之和最接近第一艘轮船。由此可知，装载问题等价于以下特殊的0-1背包问题。

$$\max \sum_{i=1}^n w_i x_i$$

$$\text{s.t. } \sum_{i=1}^n w_i x_i \leq c_1$$

$$x_i \in \{0,1\}, 1 \leq i \leq n$$

用回溯法设计解装载问题的 **$O(2^n)$** 计算时间算法，在某些情况下该算法优于动态规划算法。

装载问题

- 1) 每个集装箱要么出现要么不出现，设第 i 个集装箱是否出现表示为 x_i ，则问题解向量： $(x_1, x_2, \dots, x_n)$
- 2) 显式约束： $x_i \in \{0, 1\}$
- 3) 隐式约束： x_i 为1的集装箱总重量小于等于 c_1
- 4) 界函数：

当前载重量 cw +剩余可选集装箱的重量和 $r >$ 当前最优载重量 $bestw$ 。

装载问题—标准回溯

```
void backtrack (int t)
{
    if (t > n) output(x);
    else
        for (int i = f(n, t): g(n, t))
        {
            x[t] = h(i);
            if (constraint(t) & bound(t))
                backtrack(t+1);
        }
}
```

n 集装箱的数量;
w[n] 集装箱的重量;
bestx 最优解;
bestw 最优重量;
cw = 0 当前集装箱的重量;
void backtrack (int t)
{// 搜索第t层结点
 if (t > n) // 到达叶结点
 更新最优解bestx,bestw;
 else
 for (int i = 1; i >= 0; i--) {
 x[t] = i;
 if (cw + w[t] * i <= c) {
 cw += w[t] * i;
 backtrack(t + 1);
 cw -= w[t] * i;
 }
 }
}

装载问题—标准回溯

$r = \text{sum}(w)$; 1~n个集装箱的重量和;

n物品的数量;

bestx最优解;

bestw最优重量;

cw = 0当前物品的重量;

void backtrack (int t)

{// 搜索第t层结点

if (t > n) // 到达叶结点

更新最优解bestx, bestw;

else{

$r -= w[t]$; // t+1~n个集装箱的重量和

for(int i = 1; i >= 0; i--) {

x[t] = i;

if(cw + w[t] * i <= c &&

$cw + w[t] * i + r > \text{bestw}$) {

cw += w[t] * i;

backtrack(t+ 1);

cw -= w[t] * i; }

}

$r += w[t]$; } //向上回溯的时候得加上

}

解空间：子集树

可行性约束函数(选择当前元素):

$cw + w[t] \leq c$

上界函数:

当前载重量cw+剩余可选集装箱的重量和r > 当前最优载重量bestw

r用矩阵存储
是否可行?

装载问题

```
r=sum(w); 剩余材料的重量;  
n物品的数量;  
bestx最优解;  
bestw最优重量;  
cw当前物品的重量;  
void backtrack (int i)  
{// 搜索第i层结点  
  if (i > n) // 到达叶结点  
    更新最优解bestx, bestw, return;  
  r -= w[i];  
  if (cw + w[i] <= c) {// 搜索左子树, x[i] = 1  
    x[i] = 1;  
    cw += w[i];  
    backtrack(i + 1);  
    x[i] = 0;  
    cw -= w[i];  }  
  if (cw + r > bestw) {// 搜索右子树, x[i] = 0  
    x[i] = 0;  
    backtrack(i + 1);  }  
  r += w[i];  
}
```

解空间：子集树

可行性约束函数(选择当前元素):

$$cw + w[i] \leq c$$

上界函数:

当前载重量 cw +剩余可选集装箱的重量和 $r >$ 当前最优载重量 $bestw$

进左前: $ub = cw + r > bestw$

进左后: $ub' = cw + w[i] + r - w[i] = ub$

进右前: $cw \leq c$

进右后: $cw + 0 \leq c$

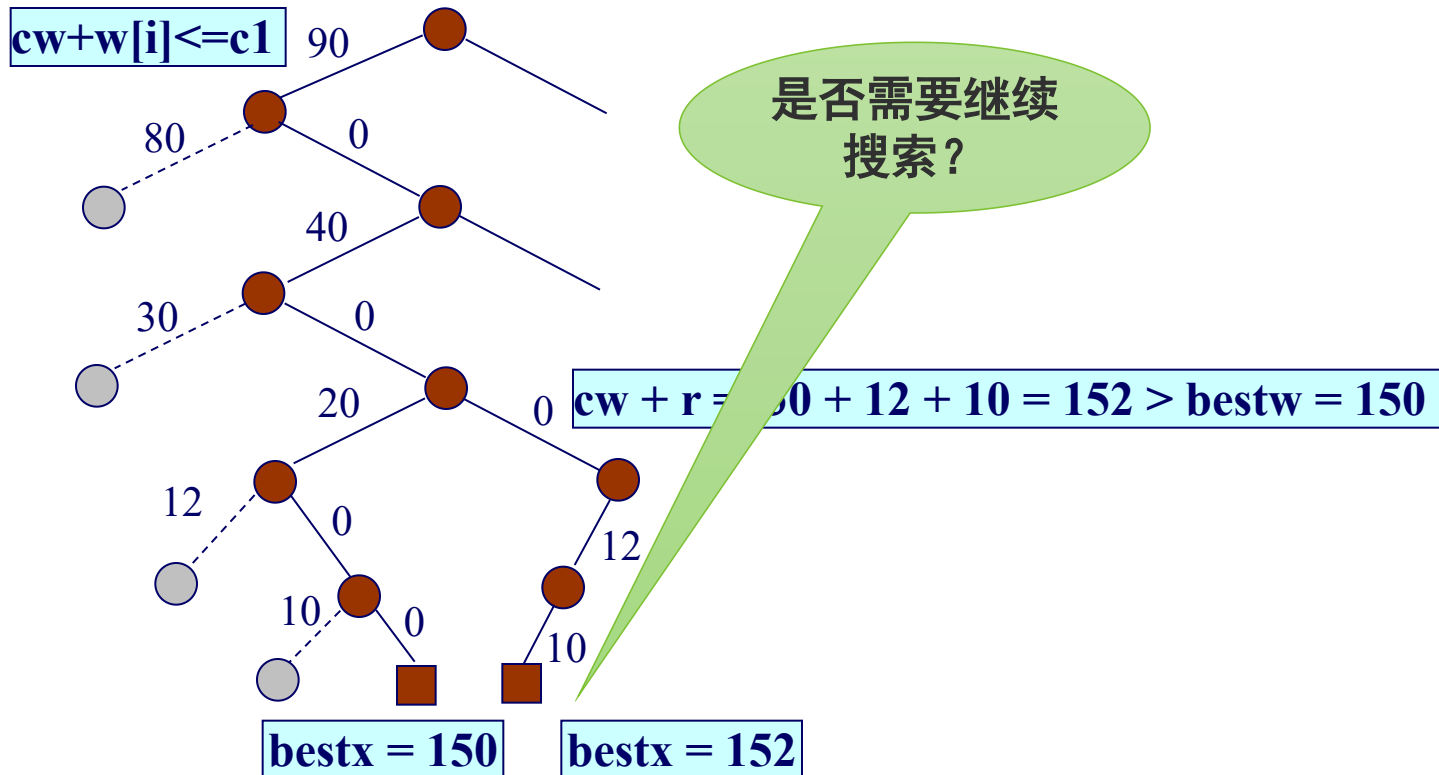
算法时间复杂度 $O(2^n)$ 。

装载问题

子集树问题

$W = \langle 90, 80, 40, 30, 20, 12, 10 \rangle$, $c_1 = 152$, $c_2 = 130$ 。

$bestx = -1$



批作业调度问题

给定 n 个作业的集合 $\{J_1, J_2, \dots, J_n\}$ 。每个作业必须先由机器1处理，然后由机器2处理。所有作业在机器2上完成处理的时间和称为该作业调度的完成时间和。

批处理作业调度问题要求对于给定的 n 个作业，制定最佳作业调度方案(给出作业的加工顺序)，使其完成时间和达到最小。

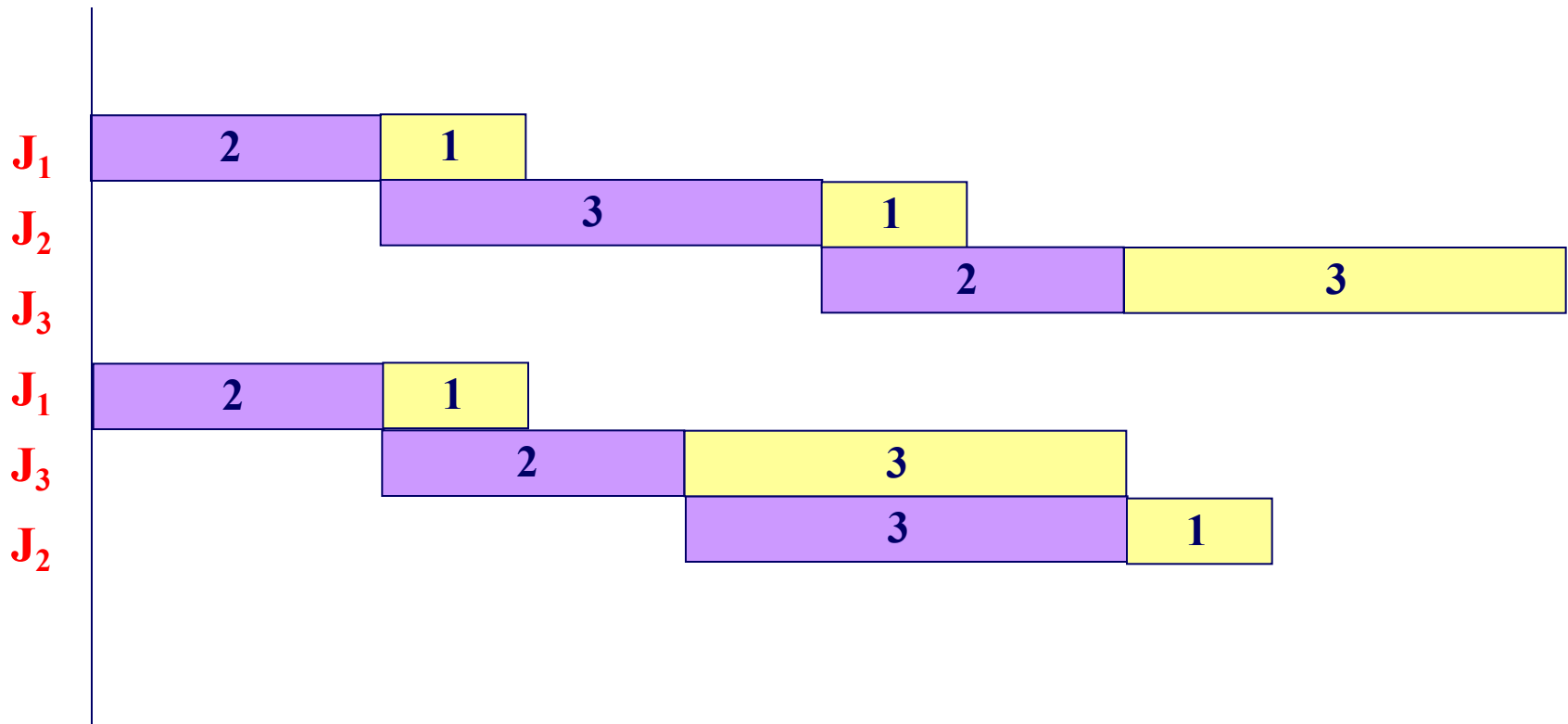
批作业调度问题

	机器1	机器2
作业1	2	1
作业2	3	1
作业3	2	3

这3个作业的6种可能的调度方案是1,2,3; 1,3,2; 2,1,3; 2,3,1; 3,1,2; 3,2,1; 它们所相应的完成时间和分别是19, 18, 20, 21, 19, 19。易见, 最佳调度方案是1,3,2, 其完成时间和为18。

批作业调度问题

	机器1	机器2
作业1	2	1
作业2	3	1
作业3	2	3



批作业调度问题

- 1) 每个工件出现且出现一次，设第 i 个出现的工件序号为 x_i ，则**问题解向量**： $(x_1, x_2, \dots, x_n)$
- 2) **显约束**： $x_i \in \{1, 2, \dots, n\}, \forall i \in \{1, 2, \dots, n\}$
- 3) **隐约束**： $x_i \neq x_k, \forall i, k \in \{1, 2, \dots, n\} \text{ 且 } i \neq k$
- 4) **界函数**:

当前工件的完成时间和**小于**当前找到的最优调度的完成时间和。

批作业调度问题

本质是一个**全排列问题**，解空间属于排列树问题，**参考货郎担代码**即可求解，难点是给定一个解，如何计算其完工时间？

假设 $\langle x_1, x_2, x_3, x_4, \dots, x_n \rangle$ 是一个调度序列，令 M_{i1} 表示工件 x_i 在第一个机器上的完成时间， M_{i2} 表示工件 x_i 在第二个机器上的完成时间， F_{1xi} ， F_{2xi} 为工件 x_i 在机器1和2上的**加工时间**，则完成时间计算过程为：

$$M_{i1} = M_{(i-1)1} + F_{1xi}; M_{i2} = \max\{M_{(i-1)2}, M_{i1}\} + F_{2xi};$$

求解过程中，如果计算出来的完成时间和**大于等于**当前找到的最优调度的完成时间和，则**剪枝**。

批作业调度问题—递归求解

```

a[n][n]; //邻接矩阵, 存储任意两个城市间的代价;
bestx[n]; //存储当前最小代价对应的路线;
bestc = MaxInt; //存储当前最小代价
cc = 0; //存储当前代价
void Traveling<Type>::Backtrack(int t){ //i的初值为2;
    if (t > n) {
        bestc = cc; bestx = x;
    }
    else {
        for (int j = t; j ≤ n; j++){
            swap(x[t], x[j]);
            if(check(x, j, t, a, n) ) {
                if(t < n && cc + a[x[t-1]][x[t]] < bestc) {
                    cc = cc + a[x[t-1]][x[t]];
                    backtrack(t + 1);
                    cc = cc - a[x[t-1]][x[t]];
                }
                if(t == n && cc + a[x[t-1]][x[t]] < bestc) {
                    cc = cc + a[x[t-1]][x[t]] - a[x[n]][[1]];
                    backtrack(t + 1);
                    cc = cc - a[x[t-1]][x[t]] - a[x[n]][[1]];
                }
            }
            swap(x[t], x[j]); //恢复现场
        }
    }
}

```

$M_{i1} = M_{(i-1)1} + F_{1xi}$
 $M_{i2} = \max\{M_{(i-1)2}, M_{i1}\}$

+ F_{2xi}

M_2 为什么
不恢复?

```

F[2][n]; //任务加工时间矩阵;
bestx[n]; //存储完成时间最短的加工顺序;
bestf = MaxInt; //存储当前最短完工时间和
cf = 0; //存储当前完成时间和
M1 = 0; //机器1的完成时间
M2[n+1] = 0; 机器2的完成时间
void Scheduling<Type>::Backtrack(int t){
    if (t > n) {
        bestf = cf; bestx = x;
    }
    else {
        for (int i = t; i ≤ n; i++){
            swap(x[t], x[i]);
            M1 += F[1][x[t]];
            M2[t] = max(M2[t-1], M1) + F[2][x[t]];
            cf += M2[t];
            if(check(bestf, cf) ) { //检查bound
                backtrack(t + 1);
            }
            M1 -= F[1][x[t]]; //恢复现场
            cf -= M2[t];
            swap(x[t], x[i]); //恢复现场
        }
    }
}

```


0-1 背包问题

解空间：子集树

可行性约束函数： $\sum_{i=1}^n w_i x_i \leq c_1$

上界函数：

装载问题的上界函数为
当前载量 cp +剩余可选
集装箱重量和 r >当前最
优装载量 $bestp$;

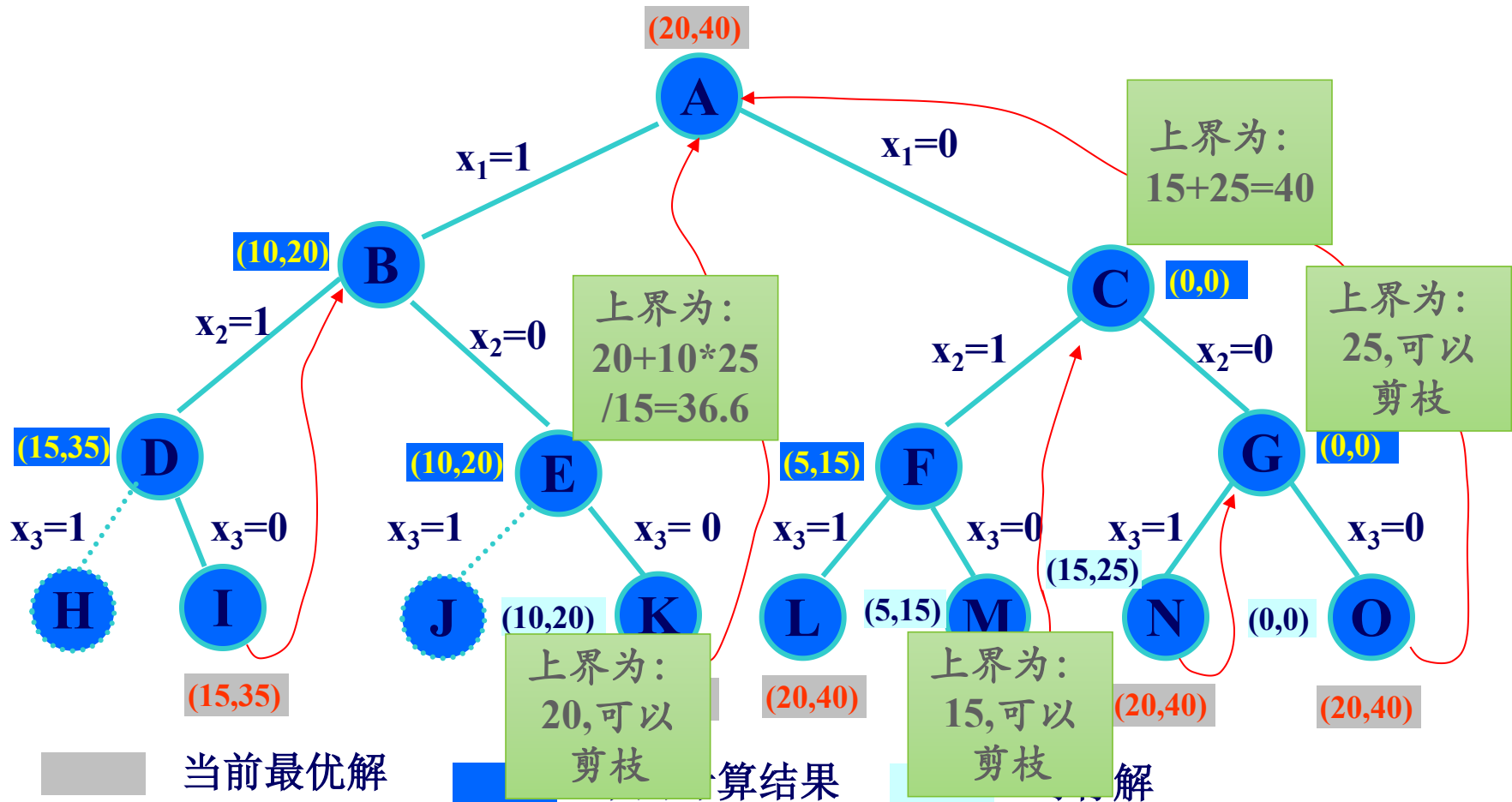
对于0-1背包问题，更好
的上界为：当前价值
 cv +剩余容量可容纳的
最大价值>当前最优价
值 $bestp$ 。

```
template<class Typew, class Typep>
Typep Knap<Typew, Typep>::Bound(int i)
{// 计算上界
    Typew cleft = c - cw; // 剩余容量
    Typep b = cv; // 当前价值
    // 以剩余物品单位重量价值递减序 装入物品
    while (i <= n && w[i] <= cleft) {
        cleft -= w[i]; // 重量减少
        b += p[i]; // 价值增加
        i++;
    }
    // 装满背包
    if (i <= n) b += p[i]/w[i] * cleft;
    return b;
}
```

思想同背
包问题的
贪心法

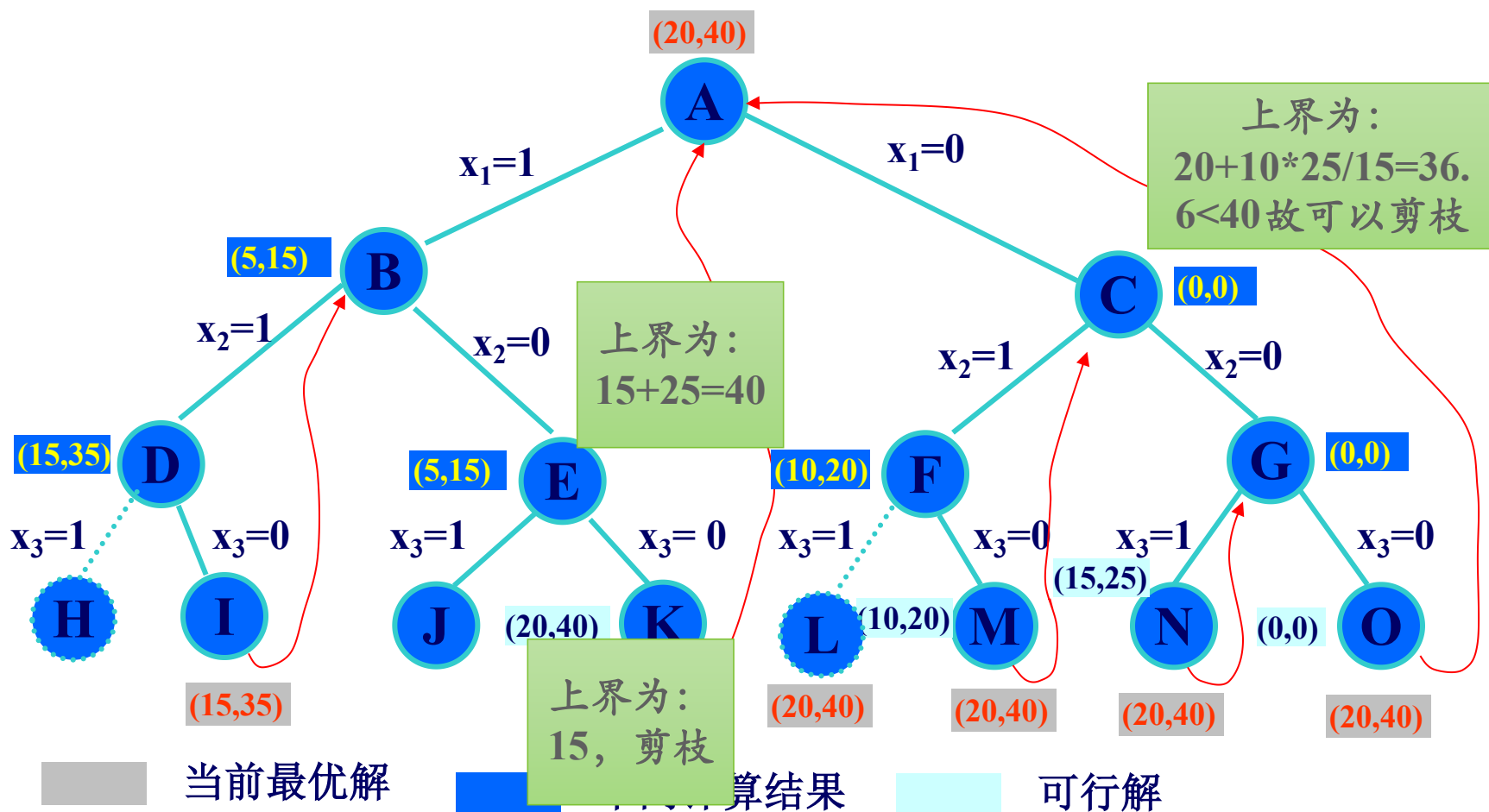
0-1背包问题

例： $n=3$ ， $C=20$ ， $(v_1, v_2, v_3)=(20, 15, 25)$ ， $(w_1, w_2, w_3)=(10, 5, 15)$ ，
求 $X=(x_1, x_2, x_3)$ 使背包价值最大？



0-1 背包问题

按单位重量价值排序后： $n=3$ ， $C=20$ ， $(v_1, v_2, v_3)=(15, 20, 25)$ ， $(w_1, w_2, w_3)=(5, 10, 15)$ ，求 $X=(x_1, x_2, x_3)$ 使背包价值最大？



最大团问题

给定无向图 $G=(V, E)$ 。如果 $U \subseteq V$ ，且对任意 $u, v \in U$ 有 $(u, v) \in E$ ，则称 U 是 G 的**完全子图**。

G 的**完全子图** U 是 G 的**团**当且仅当 U 不包含在 G 的**更大的完全子图**中。
 G 的**最大团**是指 G 中所含**顶点数最多的团**。

有些定义是团、
极大团、最大团

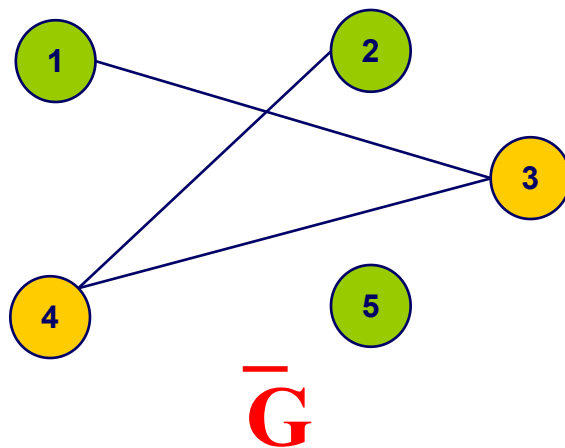
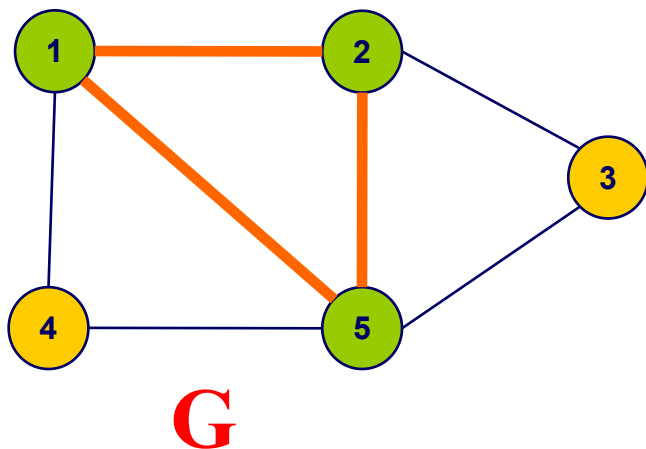
如果 $U \subseteq V$ 且对任意 $u, v \in U$ 有 $(u, v) \notin E$ ，则称 U 是 G 的**空子图**。

G 的**空子图** U 是 G 的**独立集**当且仅当 U 不包含在 G 的更大的空子图中。
 G 的**最大独立集**是 G 中所含**顶点数最多的独立集**。

对于任一无向图 $G=(V, E)$ ，其**补图** $\bar{G}=(V_1, E_1)$ 定义为： $V_1=V$ ，且 $(u, v) \in E_1$ 当且仅当 $(u, v) \notin E$ 。

U 是 G 的最大团当且仅当 U 是 $\bar{G}$ 的最大独立集。

最大团问题



(1,2) 是 G 的完全子图吗?

(1,2) 是 G 的团吗?

(1,2) 是 $\bar{G}$ 的空子图吗?

(1, 2, 5) 是 G 的完全子图吗?

(1, 2, 5) 是 G 的团吗?

(1, 2, 5) 是 $\bar{G}$ 的空子图吗?

无向图 G 的最大团和最大独立集问题可以看作是图 G 的顶点集 V 的子集选取问题。最大团就是包含顶点最多的完全子图。

最大团问题

- 1) 每个顶点要么出现要么不出现, 设第 i 个顶点是否出现表示为 x_i , 则问题解向量: $(x_1, x_2, \dots, x_n)$
- 2) 显约束: $x_i = 0$ 或者 1
- 3) 隐约束:
 - (1) 所有 $x_i = 1$ 的顶点都相互连接;
- 4) 界函数:

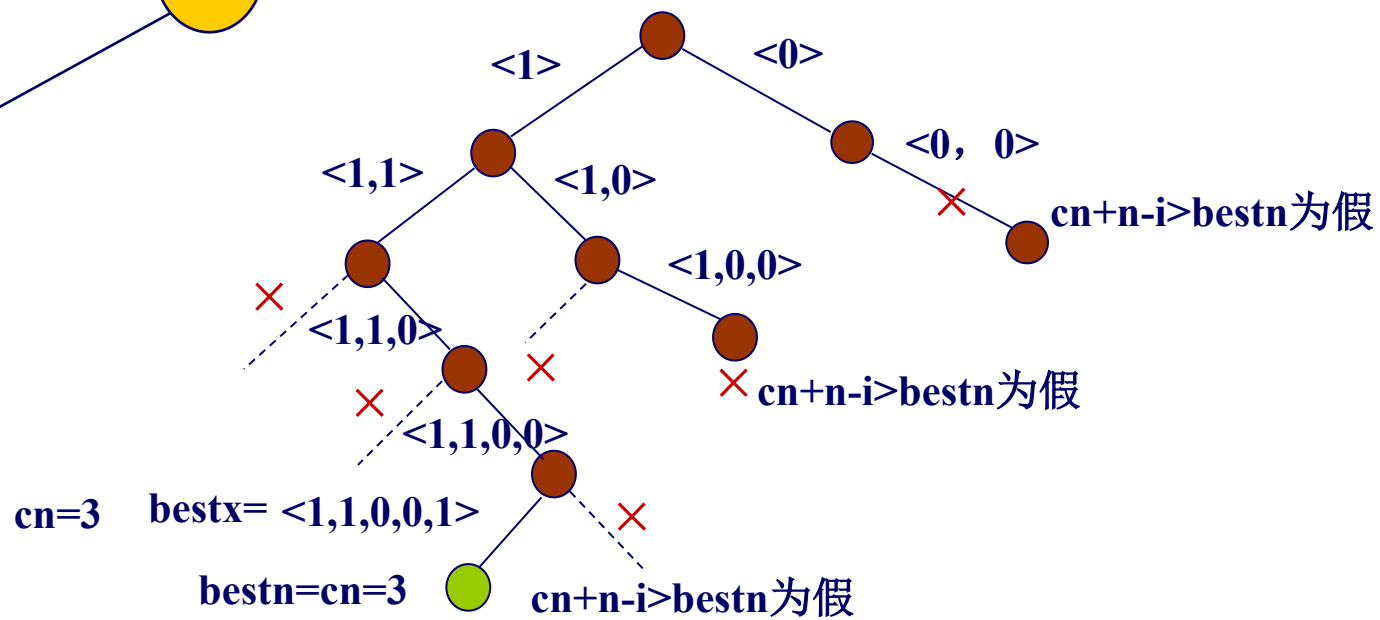
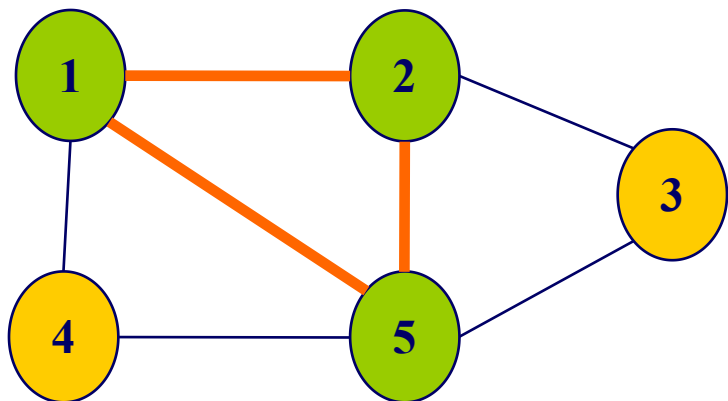
有足够多的可选择顶点使得算法有可能在右子树中找到更大的团。

最大团问题

```
cn = 0; // 当前结点数
bestx[]; // 最大团的选择方案
void Clique::Backtrack(int i){// 计算最大团
    if (i > n) {// 到达叶结点
        for (int j = 1; j ≤ n; j++) bestx[j] = x[j];
        bestn = cn; return; }
    // 检查顶点 i 与当前团的连接
    if (check(i)) {// 进入左子树, 只check不bound
        x[i] = 1;
        cn++;
        Backtrack(i+1);
        x[i] = 0;
        cn--;}
    if (cn + n - i > bestn) {// 进入右子树, 先判断界
        x[i] = 0;
        Backtrack(i+1);}
}
```

```
r=sum(w); 剩余材料的重量;
void backtrack (int i)//计算最大装载
{// 搜索第i层结点
    if (i > n) // 到达叶结点
        更新最优解bestx, bestw, return;
    r -= w[i];
    if (cw + w[i] ≤ c) {// 搜索左子树
        x[i] = 1;
        cw += w[i];
        backtrack(i + 1);
        x[i] = 0;
        cw -= w[i];    }
    if (cw + r > bestw) {
        x[i] = 0; // 搜索右子树
        backtrack(i + 1);    }
    r += w[i];
}
```


最大团问题



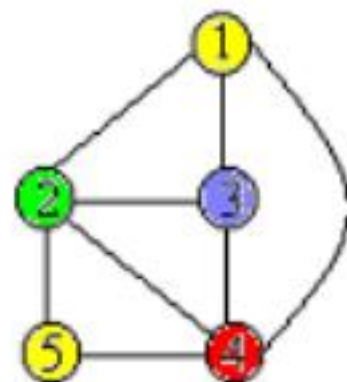
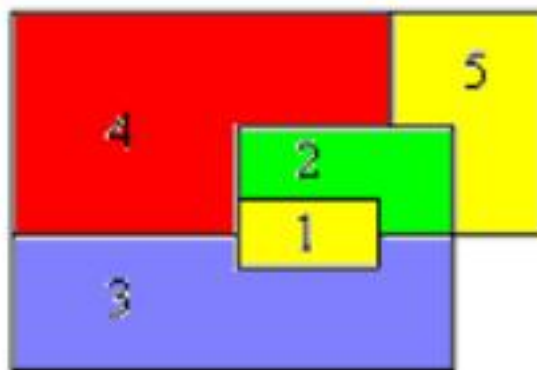
图的 m 着色问题

给定无向连通图 G 和 m 种不同的颜色。用这些颜色为图 G 的各顶点着色，每个顶点着一种颜色。是否有一种着色法使 G 中每条边的2个顶点着不同颜色。

这个问题是**图的 m 可着色判定问题**。若一个图**最少需要 m 种颜色**才能使图中每条边连接的2个顶点着不同颜色，则称这个数 **m 为该图的色数**。

求一个图的色数 m 的问题称为图的 m 可着色优化问题。

平面图的四色猜想。



图的m着色问题

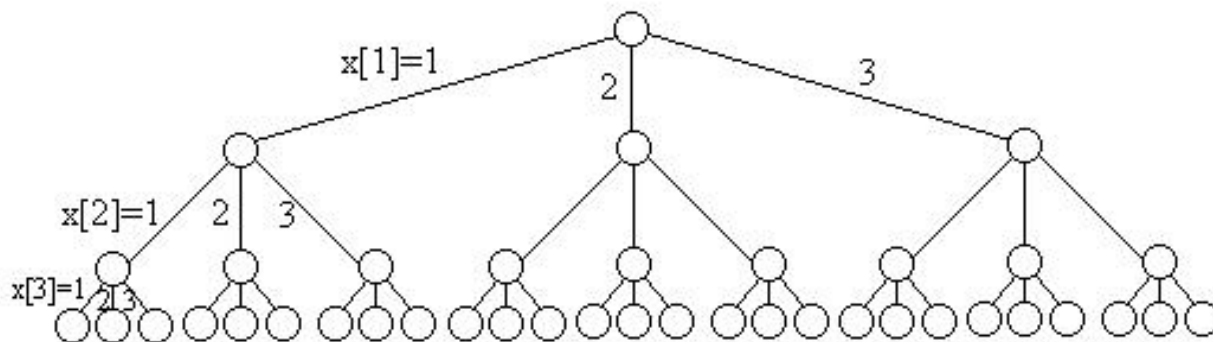
对于含有 n 个结点的图，问能否能用 m 种颜色着色，使得任意两个相邻的结点的着色不同？如果可以，给出所有的可行方案。

解向量： $(x_1, x_2, \dots, x_n)$ ， x_i 表示结点 i 所着颜色

显式约束： $x_i \in \{1, 2, \dots, m\}$

隐式约束函数： 所有已着色的相邻结点颜色不重复。

问题解空间可表示为一颗**高度为 $n+1$ 的完全 m 叉树**。解空间树的第 i ($1 \leq i \leq n$) 层每个结点都有 m 个儿子，每个儿子相应于 $x[i]$ 的 m 个可能着色之一。第 $n+1$ 层结点均为叶结点。



图的m着色问题

```
void Color::Backtrack(int t) //第t个结点着色
{
    //color
    if (t > n) {
        sum++;
        for (int i = 1; i <= n; i++)
            cout << x[i] << ' ';
        cout << endl;
    }
    else
        for (int i = 1; i <= m; i++) {
            x[t] = i;
            if (Ok(t)) Backtrack(t + 1);
            x[t] = 0;
        }
}

bool Color::Ok(int t)
{
    // 检查颜色重复性
    for (int j = 1; j <= n; j++)
        if ((a[t][j] == 1) && (x[j] == x[t])) return false;
    return true;
}
```

是否需要x[t]=0?

是否需要到n?

```
int n = *; sum = 0; a[n];
void Queen::Backtrack(int t)
{
    //NQueen
    if (t > n) sum++;
    else
        for (int i = 1; i <= n; i++) {
            x[t] = i;
            if (check(t)) Backtrack(t + 1);
        }
}
```

```
check(int pos) {
    for (int i = 1; i < pos; i++)
        if (abs(x[i] - x[pos]) == abs(i - pos) or x[i] == x[pos])
            return 0;
    return 1; }
```

图的m着色问题

```
void Color::Backtrack(int t)
```

```
{
```

```
    if (t>n) {
```

```
        sum++;
```

```
        for (int i=1; i<=n; i++)
```

```
            cout << x[i] << ' ';
```

```
        cout << endl;
```

```
    }
```

```
    else
```

```
        for (int i=1; i<=m; i++) {
```

```
            x[t]=i;
```

```
            if (Ok(t)) Backtrack(t+1);
```

```
            x[t]=0;
```

```
        }
```

```
}
```

```
bool Color::Ok(int t)
```

```
{// 检查颜色重复性
```

```
    for (int j=1; j<=n; j++)
```

```
        if ((a[t][j]==1)&&(x[j]==x[t])) return false;
```

```
    return true;
```

```
}
```

并不需要x[t]=0，向下会重新计算，回溯值不会影响

并不需要check到n，check到t-1即可

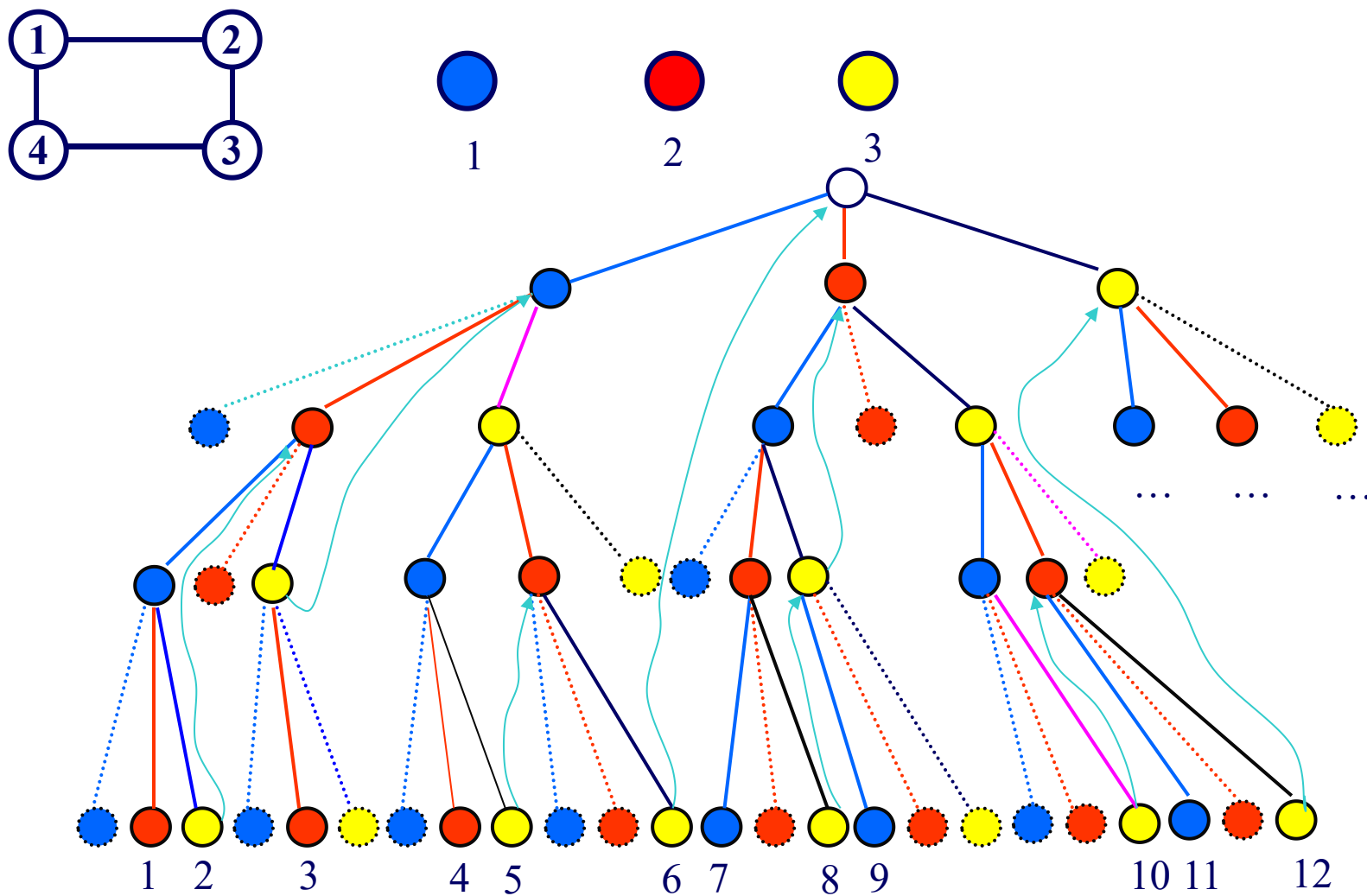
算法时间复杂度分析：

(1)结点数：

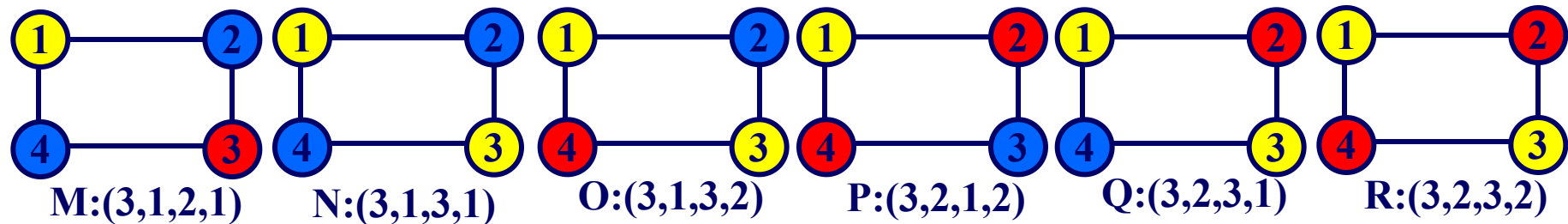
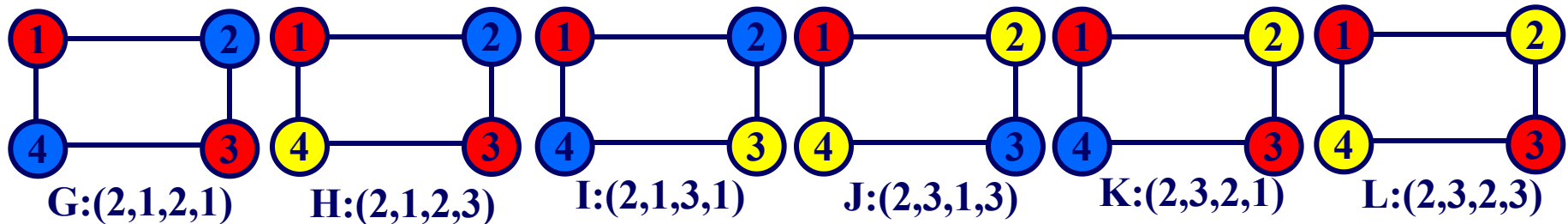
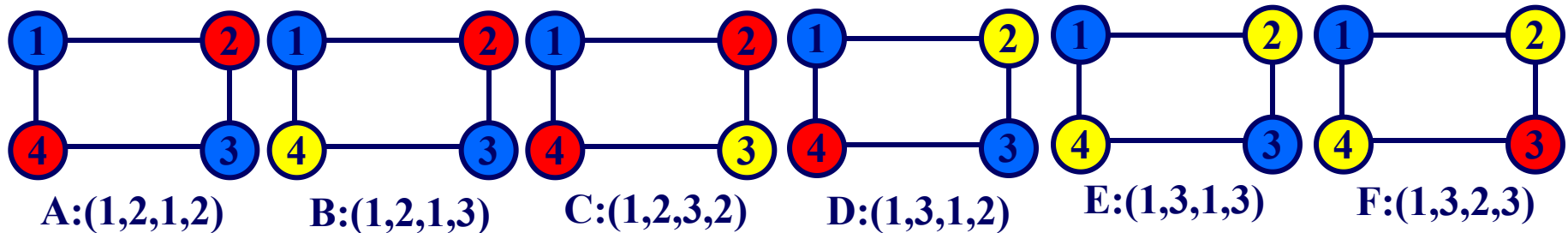
$$1+m+m^2+\dots+m^n=(m^{n+1}-1)/(m-1)\leq m^{n+1}/(m/2)=2m^n$$

(2)最坏情况下每个结点都要与其它所有结点的颜色比较，复杂度O(n)；
故复杂度为O(nmⁿ)

图的m着色问题



图的m着色问题



一个有趣的高精度数

一个有趣的高精度数据：构造一个尽可能大的数，使其从高到低满足前一位能被1整除，前2为能被2整除，...，前 n 位能被 n 整除。

高精度数据如何保存？

数组

数学模型：记高精度数为 $a_1, a_2, a_3, \dots, a_n$ 有两个要求：

- (1) a_1 能被1整除,且 $(10 a_1 + a_2)$ 能被2整除,依此类推;
- (2) 求最大的这样的数。

一个有趣的高精度数

```
int[] a = new int[bigInt]; // 存储当前值
int[] b = new int[bigInt]; // 存储最优值

public void backTracking4MaxNumber(int t) {
    if(bigThan(a, b)) { // 每找到一个需要比较
        assignA2B(a, b);
    }
    int j;
    if(t == 1) j = 1; // 第一位不能取0
    else j = 0; // 其它位可以
    for (; j <= 9; j++) {
        a[t] = j;
        if (OK(a, t)) {
            backTracking4MaxNumber(t + 1);
        }
        a[t] = -1;
    }
}
```

```
public boolean OK(int[] a, int t)
{
    int r = 0;
    for(int i = 1; i <= t; i++) {
        r = r * 10 + a[i];
        r = r % t;
        // 整数倍乘以10没用故只用余数, 防止溢出
    }
    r = r % t;
    if(r == 0) return true;
    return false;
}
```

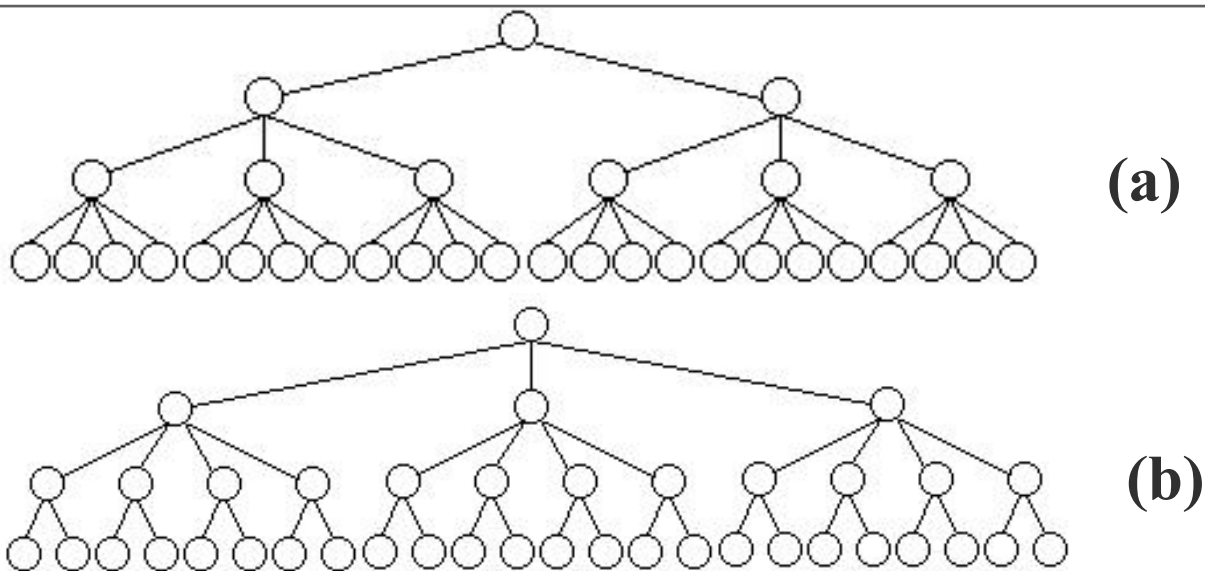

回溯法效率分析

通过前面具体实例的讨论容易看出，回溯算法的效率在很大程度上依赖于以下因素：

- (1)产生 $x[k]$ 的时间；
- (2)计算约束函数**constraint**的时间；
- (3)计算上界函数**bound**的时间；
- (4)满足约束和界函数约束的所有 $x[k]$ 的个数。

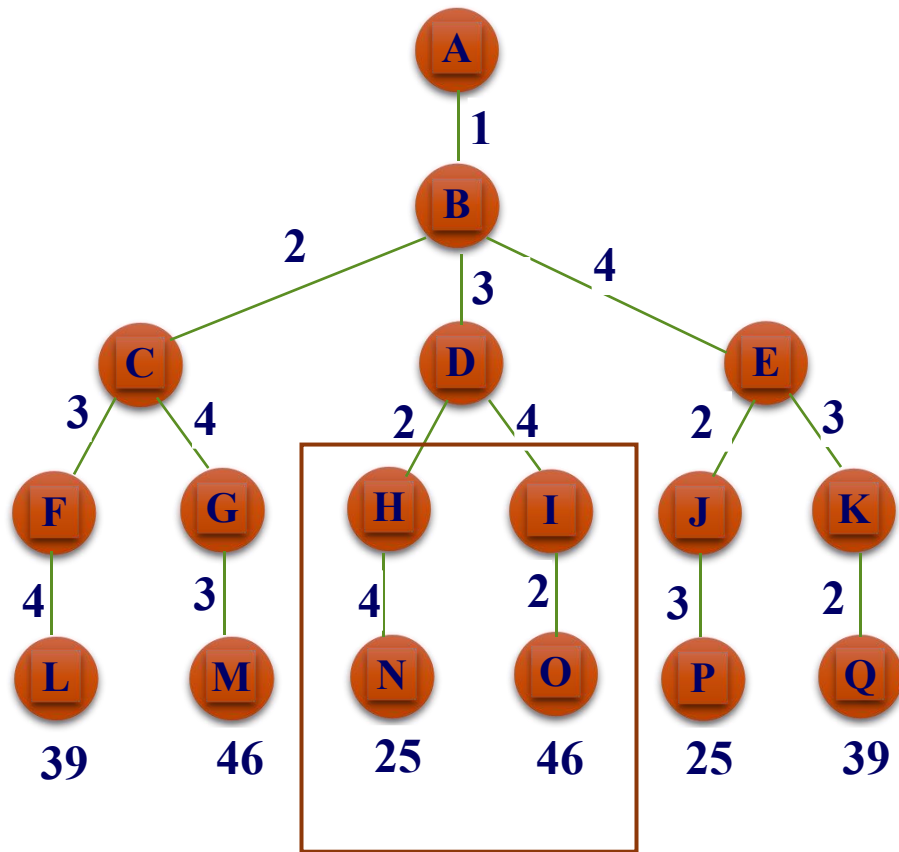
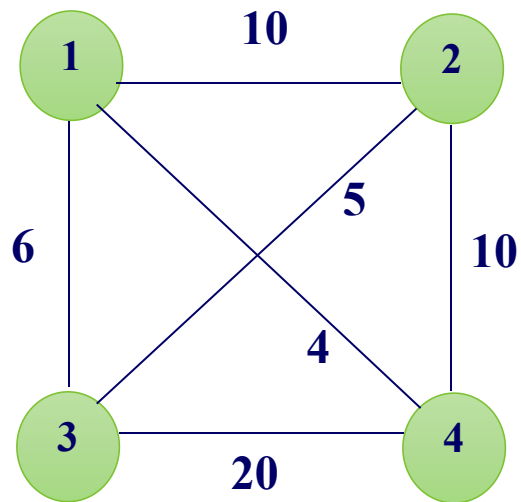
回溯法效率分析

对于许多问题而言，在搜索试探时选取 $x[i]$ 的值顺序是任意的。
在其他条件相当的前提下，让可取值最少的 $x[i]$ 优先。从图中关于同一问题的2棵不同解空间树，可以体会到这种策略的潜力。



图(a)中，从第2层剪去1棵子树，则从所有应当考虑的叶结点中一次消去12个叶结点。对于图(b)，虽然同样从第1层剪去1棵子树，却只从应当考虑的叶结点中消去8个叶结点。前者的效果明显比后者好。

回溯法启发

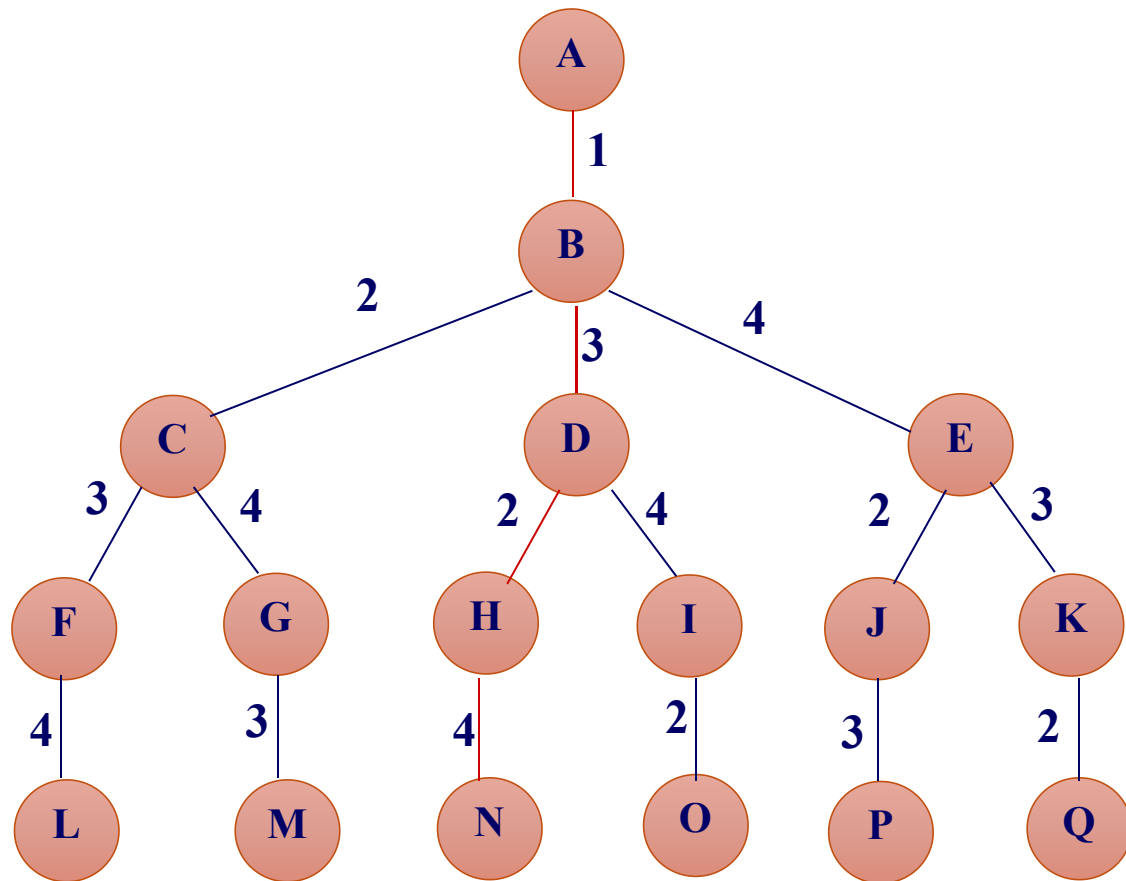


总结

- 适应于求解组合搜索问题（含组合优化问题）
- 求解条件：满足多米诺性质
- 解的表示：解向量，求解是不断扩充解向量的过程
- 回溯条件：约束条件或者界条件
- 算法复杂性：最坏情况为指数，空间代价小
- 降低时间复杂性的主要途径：利用界函数对搜索树进行剪枝。

讨论

- 问题一：回溯法采用的深度优先的搜索方式能够快速找到一个解，但是如果解空间太大，那么无论怎么回溯都不能回到很浅的层次，这会导致什么问题？
- 答：解空间太大时导致的问题：很难回到根结点，探索前面的步骤的其它走法；如果前面走得不好，那么再怎么搜索都不会得到好的解。



讨论

- 问题二：如何能够避免该问题？
- 答：可以采用函数优先的选点搜索方式，确保前面搜索的解的质量；或者函数和深度优先结合的方式，在保证搜索结点的的质量的同时，加快搜索进度；或者循环深度优先的方式，不断从根结点重新尝试新的搜索路径。

如果不恢复现场则可能出现这样的问题

例如 $x = \{, 1, 2, 3, 4, 5\}$

- 将2-5依次交换到 $x[2]$ 进行搜索，例如得到了 $x = \{, 1, 3, 2, 4, 5\}$
- 搜索 $x = \{, 1, 3, 2, 4, 5\}$ 过程中，可能最后得到 $x = \{, 1, 3, 4, 2, 5\}$
- 1,3开头的搜索完后，希望搜索1,4开头的，但是交换 $x[4]$ 到 $x[2]$ 得到 $x = \{, 1, 2, 4, 3, 5\}$ ，显然1,2开头的搜索过了，这样部分搜索会重复，部分搜索没有搜索到，所以必须恢复现场。

如果恢复现场，则搜索 $x = \{, 1, 3, 2, 4, 5\}$ 以1,3开头的搜索结束后，会执行恢复现场，变成 $x = \{, 1, 2, 3, 4, 5\}$ ，然后将交换 $x[4]$ 到 $x[2]$ 得到 $x = \{, 1, 4, 3, 2, 5\}$ ，进而搜索1,4开头的空间。

